

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



Code of Conduct:

I Sanjay Raghavendra/ 192411041 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS:

1. Testing as an Engineering Activity in a Banking Application

Answer:

Testing should be treated as a systematic engineering activity rather than an informal or ad-hoc process. Since the banking application is facing frequent failures, the team should introduce a well-defined testing process.

First, the team should perform systematic test design based on requirements. Techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, and Risk-Based Testing can be used. Test cases should contain test conditions, input data, expected results, actual results, and pass/fail status.

Testing tools can be introduced for test management, defect tracking, automation, and continuous integration. Standards should define test case formats, defect severity, reporting procedures, entry criteria, and exit criteria.

For example, an online banking transfer should be tested for sufficient balance, insufficient balance, invalid account number, maximum transfer limit, duplicate transaction, and network failure.

Instead of random testing, the team should use planned, risk-based, regression, exploratory, and automated testing.

Thus, engineering-based testing provides better test coverage, repeatability, early defect detection, traceability, and overall banking application quality.

2. Structured Testing Process for an ATM System

Answer:

An ATM requires a structured testing process because financial transactions must be accurate and reliable. Testing should be performed throughout the SDLC.

Stages of testing:

1. Requirement Analysis: Identify requirements such as valid PIN, account validation, balance checking, withdrawal limits, cash availability, and receipt generation.
2. Test Planning: Define testing scope, resources, environment, schedule, risks, and entry/exit criteria.
3. Test Case Design: Prepare positive and negative test cases.
4. Test Execution: Perform unit, integration, system, and regression testing.
5. Defect Reporting: Record defects with steps to reproduce, expected result, actual result, severity, and evidence.
6. Retesting and Closure: Verify fixes and perform regression testing before final release.

For example, when a customer withdraws ₹5,000, the ATM should verify the PIN, account balance, withdrawal limit, cash availability, and correct deduction from the account.

A consistent testing process prevents important scenarios from being missed and ensures every module is tested using the same standards.

Therefore, structured testing results in fewer defects, reliable transactions, improved security, and better ATM software quality.

3. Error, Defect and Failure in a Web Application

Answer:

Error, defect, and failure are related but different concepts in software testing.

- Error: A human mistake made during requirements, design, coding, or testing.
- Defect: A fault introduced into the software because of an error.
- Failure: The observable incorrect behavior produced when the defect is executed.

For example, suppose a developer incorrectly implements the login condition. The developer's programming mistake is an error. The incorrect login logic present in the software is a defect. When a valid user enters the correct username and password but cannot log in, the user experiences a failure.

Similarly, in a banking transaction, if the expected amount is ₹5,000 but the application deducts ₹10,000, the incorrect code is the defect and the wrong deduction observed by the customer is the failure.

If testers misunderstand these terms, defect reports may be unclear and developers may find it difficult to identify the actual cause.

Therefore, reports should clearly mention module, steps to reproduce, expected result, actual result, severity, priority, environment, screenshots, and logs. This improves communication and speeds up defect resolution.

4. Defect Clustering and Test Case Prioritization

Answer:

Defect clustering is a software testing principle which states that a small number of modules often contain a large number of defects. Therefore, testing should focus more attention on defect-prone and high-risk areas. The testing team should analyze previous defect data and identify modules with frequent failures. Test cases can then be prioritized based on:

- Number of previous defects
- Business importance
- Risk level
- Complexity
- Frequency of use
- Recent code changes

- Security and financial impact

For example, in an e-commerce application, suppose the payment module contains most previous defects.

The tester should prioritize payment-related test cases before less critical UI features.

Testers can use risk-based testing, regression testing, boundary testing, and exploratory testing for high-risk modules.

Defect clustering helps the team use limited testing time effectively. It does not mean that other modules should be ignored; rather, more testing effort should be given to areas where defects are more likely to occur. Thus, applying defect clustering improves test prioritization, defect detection, testing efficiency, risk management, and overall software quality.

5. Defect Classification and Defect Repository

Answer:

A defect repository is a centralized collection of information about software defects found during testing and production. It helps the organization learn from previous failures and prevent their recurrence.

Defects may originate from:

- Incorrect requirements
- Design mistakes
- Coding errors
- Interface or integration problems
- Database problems
- Configuration/environment issues
- Testing mistakes

They can be categorized according to type, severity, priority, module, root cause, status, and release.

For example, if several releases contain login defects caused by incorrect password validation, the defect repository will reveal a recurring pattern. Testers can use this information to create additional login validation and regression test cases.

The repository should contain the defect description, steps to reproduce, expected and actual results, root cause, resolution, and affected version.

The team should perform root-cause analysis, improve coding and review practices, and add recurring defects to regression test suites.

Therefore, effective use of a defect repository helps in defect prevention, better test planning, faster analysis, improved regression testing, and higher software quality.

6. Mobile Banking Application Crashing During Fund Transfer

Answer:

A mobile banking application crashing during fund transfer is a critical problem because it can affect both customers and financial transactions. The tester should use a systematic approach to identify the root cause. First, the tester should reproduce the crash and collect application logs, error messages, stack traces, device information, network conditions, and transaction details.

Important test scenarios include:

- Successful fund transfer
- Insufficient balance
- Invalid beneficiary
- Invalid account details
- Maximum transfer amount
- Duplicate transfer
- Network interruption

- Session timeout
- Server failure
- Concurrent transactions

For example, if the application crashes when the network connection is lost during a transfer, the tester should check whether the application properly handles the exception and whether the transaction is rolled back safely.

After the developer fixes the problem, the tester should perform retesting followed by regression testing. Future failures can be reduced using automated regression testing, negative testing, boundary testing, integration testing, exception handling, and continuous monitoring.

This systematic approach improves application stability and prevents critical banking failures from reaching customers.

7. Testing and Debugging

Answer:

Testing and debugging are different but complementary software quality activities.

Testing is the process of executing software to identify failures and determine whether it satisfies specified requirements. Debugging is the process of finding the root cause of a defect and correcting the source code or related configuration.

Testing

Identifies failures Finds the cause of failures

Mainly performed by testers Mainly performed by developers

Compares expected and actual results Examines code, logic, data, and execution

Produces defect reports Produces corrective changes

For example, if an ATM deducts ₹10,000 instead of ₹5,000, the tester identifies and reports the failure. The developer then debugs the transaction calculation and identifies the incorrect logic.

Confusing testing with debugging can result in poor defect reporting and inefficient resolution.

A proper workflow should be:

Testing → Failure Detection → Defect Reporting → Debugging → Fix → Retesting → Regression Testing

This separation allows testers to focus on finding defects while developers focus on correcting their causes, resulting in better software quality.

8. Cross-Browser Testing for an E-Commerce Platform

Answer:

Cross-browser testing ensures that a web application works correctly across different web browsers, operating systems, screen sizes, and browser versions.

First, the testing team should identify the browsers supported by the application, such as Chrome, Firefox, Edge, and Safari. A browser compatibility matrix can then be created.

Important features such as login, product search, shopping cart, checkout, payment, navigation, and forms should be tested across the selected environments.

Defects may occur because of:

- Different browser rendering engines
- JavaScript compatibility issues
- CSS differences
- Unsupported browser features
- Responsive design problems
- Browser-specific APIs

For example, a checkout button may work correctly in Chrome but fail in Firefox because of

browser-specific JavaScript behavior.

Tools such as Selenium, Playwright, or browser testing platforms can automate repeated cross-browser tests. Systematic cross-browser testing improves compatibility, consistency, reliability, user experience, and test coverage, ensuring that customers receive similar functionality regardless of the browser they use.

9. Payroll Logical Errors and Defect Classification

Answer:

Incorrect salary calculation caused by incorrect program logic is primarily a logic or calculation defect.

Defect classification helps testers understand the nature and importance of the problem.

The defect can be classified as:

- Defect type: Logic/calculation defect
- Module: Payroll/salary calculation
- Severity: High
- Priority: High
- Root cause: Incorrect implementation of salary rules

For example, the correct formula may be:

$\text{Net Salary} = \text{Basic Salary} + \text{Allowances} - \text{Deductions}$

If the program accidentally adds deductions instead of subtracting them, employees will receive incorrect salaries.

Testers should test different employee categories, salaries, tax values, allowances, deductions, overtime, zero values, and boundary values.

Useful techniques include equivalence partitioning, boundary value analysis, decision table testing, unit testing, code review, and regression testing.

Understanding defect classes helps testers select suitable test techniques and prioritize serious problems.

Therefore, proper classification combined with systematic testing can prevent repeated payroll errors and improve the accuracy, reliability, and maintainability of the payroll system.

10. Structured Test Case Design Instead of Random Testing

Answer:

Random testing without proper planning can miss important scenarios. Testing should therefore be redesigned using structured test case design.

A good test case should include:

1. Test case ID
2. Requirement/module
3. Preconditions
4. Test data
5. Test steps
6. Expected result
7. Actual result
8. Pass/Fail status

Testers should use systematic techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Risk-Based Testing

For example, suppose an application accepts an age from 18 to 60. Instead of randomly testing ages,

boundary testing should include 17, 18, 19, 59, 60, and 61.

Structured test cases ensure that requirements are properly covered and that tests can be repeated after software changes.

They also provide traceability, better defect detection, measurable coverage, repeatability, and easier regression testing.

Therefore, replacing random testing with engineering-based test design makes the testing process more reliable, organized, and effective.

11. Data Inconsistency in a Healthcare Application

Answer:

Data inconsistency occurs when different modules of a healthcare application display or store different values for the same patient information. This can be caused by incorrect requirements, database mapping errors, integration defects, validation problems, synchronization issues, or concurrency problems.

The tester should first identify where the inconsistent data originates. Then the following testing should be performed:

- Unit testing of individual modules
- Integration testing between modules
- Database validation
- Data consistency testing
- API testing
- Concurrent update testing
- Regression testing

For example, if a patient's updated address is correct in the registration module but incorrect in the billing module, the tester should check the interface or database synchronization between the two modules.

Every defect should be documented with module name, input data, steps to reproduce, expected result, actual result, severity, screenshots/logs, and root cause.

Systematic testing ensures that patient information remains accurate across all modules.

This improves data integrity, reliability, traceability, communication, and patient-data quality, which are especially important in healthcare applications.

12. Repeated Defects in Software Releases

Answer:

Repeated defects indicate that the organization is not effectively learning from previous testing results. A defect repository can be used to identify and prevent these recurring problems.

The repository should record:

- Defect ID
- Module
- Defect type
- Severity
- Priority
- Root cause
- Steps to reproduce
- Resolution
- Release/version
- Current status

Testers can analyze this information to identify patterns. For example, if most previous defects occurred in the payment module, the team should increase test coverage for payment functions.

Historical defect information can be converted into new regression test cases. Repeated failures should also undergo root-cause analysis to determine whether the underlying problem is related to requirements, design, coding, or testing.

Preventive measures include code reviews, better requirements, additional regression tests, automation, developer-test collaboration, and improved test design.

Thus, a defect repository should not simply store defects; it should act as a knowledge base for improving future testing and preventing recurring defects.

13. Web-Based Learning System Failing During Peak Usage

Answer:

A web-based learning system failing during peak usage indicates that important performance and load-related scenarios were not properly tested.

Possible testing gaps include:

- Insufficient load testing
- No stress testing
- Lack of concurrency testing
- Unrealistic test data
- No scalability testing
- Inadequate server/resource monitoring
- Testing only with a small number of users

The test plan should define expected number of concurrent users, response time, throughput, resource utilization, and acceptable performance limits.

For example, if the system normally supports 1,000 users but crashes when 5,000 students access it simultaneously during an examination, load testing should have identified this limitation before deployment. Testing techniques such as load testing, stress testing, endurance testing, scalability testing, and performance monitoring should be included.

Proper test planning ensures that realistic peak-use scenarios are covered.

Therefore, better performance testing improves availability, scalability, reliability, response time, and user satisfaction, especially during important events such as online examinations.

14. Early Involvement of Testers in the SDLC

Answer:

Testers should be involved from the early stages of software development rather than waiting until coding is completed. Early involvement helps identify defects when they are cheaper and easier to fix.

Tester involvement:

- Requirement phase: Identify incomplete, ambiguous, or conflicting requirements.
- Design phase: Review workflows and identify potential risks.
- Development phase: Prepare test cases and participate in reviews.
- Testing phase: Execute functional and non-functional tests.
- Maintenance: Perform regression testing after changes.

For example, in an ATM system, a tester involved during requirement analysis may identify the need to define behavior for invalid PIN, insufficient balance, withdrawal limits, unavailable cash, and network failure.

If these cases are identified only after development, fixing them may require more time and cost.

The organization should encourage requirement reviews, design reviews, continuous communication, shared defect tracking, and collaborative test planning.

Thus, early tester involvement supports defect prevention, better requirements, improved test coverage,

reduced rework, and higher software quality.

15. Incomplete Test Coverage

Answer:

When a system produces correct results for some cases but fails for others, it may indicate incomplete test coverage. Testing only normal scenarios is insufficient because real users may provide many different inputs and conditions.

Test coverage should include:

- Positive test cases
- Negative test cases
- Boundary conditions
- Different input classes
- Error conditions
- Integration scenarios
- Regression scenarios
- Important user workflows

For example, a banking application may successfully process a ₹1,000 transfer but fail when the amount is ₹0, the maximum allowed value, greater than the available balance, or submitted during a network failure.

Testers can improve coverage using:

Equivalence Partitioning → Boundary Value Analysis → Decision Tables → State Transition Testing → Risk-Based Testing

Requirements should also be mapped to test cases using a requirements traceability matrix.

Although complete testing is generally impossible, systematic coverage of important requirements and risks significantly reduces missed defects.

Therefore, proper coverage improves reliability, confidence, validation quality, and defect detection.]

16. Improving Defect Reporting in a Logistics Application

Answer:

Poor defect reporting can delay defect identification because developers may not have enough information to reproduce or understand the issue.

A structured defect report should contain:

1. Defect ID
2. Short and clear title
3. Module
4. Environment/version
5. Preconditions
6. Steps to reproduce
7. Expected result
8. Actual result
9. Severity and priority
10. Screenshots, logs, or other evidence

For example, instead of reporting:

“Delivery is not working.”

A better report would be:

“When a valid delivery address is entered and the Continue button is selected, the application displays a blank page. Expected result: the application should navigate to the payment page.”

Clear definitions of error, defect, and failure also improve communication between testers and developers.

The team should use a standardized defect template and a defect management system. This improves traceability, communication, prioritization, reproducibility, resolution speed, and overall defect management quality.

17. Process-Based Testing for Consistency

Answer:

When different testers use different methods across modules, testing becomes inconsistent. A process-based testing approach can solve this problem by defining a common testing procedure.

A standardized process can be:

Requirement Analysis → Test Planning → Test Design → Environment Setup → Test Execution → Defect Reporting → Retesting → Regression Testing → Test Closure

Every module should follow common standards for:

- Test case format
- Test data
- Defect severity
- Defect priority
- Entry and exit criteria
- Test execution
- Reporting

For example, testers working on the login, payment, and order modules should use the same test-case format and defect-reporting procedure.

Checklists, peer reviews, test management tools, reusable test cases, and automation can further improve consistency.

Standardization prevents testers from skipping important activities and makes testing results easier to compare across modules.

Therefore, process-based testing provides repeatability, consistency, better coverage, traceability, quality control, and reliable testing results.

18. Untested Edge Cases and Boundary Testing

Answer:

Edge cases are extreme or unusual conditions that may not be tested when testers focus only on normal scenarios. Missing these cases can result in software failures after deployment.

Boundary Value Analysis (BVA) is an effective technique because many defects occur at or near the boundaries of input ranges.

For example, if a system accepts an age from 18 to 60, the tester should check:

- 17 – below the boundary
- 18 – minimum boundary
- 19 – just above minimum
- 59 – just below maximum
- 60 – maximum boundary
- 61 – above maximum

Other edge cases include zero, null values, empty fields, negative values, maximum file size, maximum transaction amount, timeout, and network interruption.

These cases should be included during test case design and later added to the regression suite.

Boundary testing increases the probability of detecting defects that normal testing may miss.

Therefore, systematic edge-case testing improves coverage, robustness, reliability, stability, and confidence before deployment.

19. Defect Documentation and Repository

Answer:

A company without proper defect documentation will find it difficult to track issues, identify recurring problems, and measure software quality. A structured defect repository can solve this problem.

Each defect should have a unique record containing:

- Defect ID
- Title and description
- Affected module
- Version/build
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshots/logs
- Root cause
- Resolution
- Current status

For example, if a login defect occurs in three different releases, the repository can show that the same authentication component is repeatedly producing failures.

Repository data can be analyzed to identify defect trends, high-risk modules, common defect types, recurring root causes, and release-wise defect patterns.

The team should regularly review historical defects and convert important recurring problems into regression test cases.

Proper documentation improves traceability, communication, analysis, defect prevention, regression testing, and future test planning.

20. Redesigning Testing for Repeated Banking Transaction Failures

Answer:

Repeated transaction failures indicate that the banking system is not being tested systematically. The testing approach should be redesigned using a structured and risk-based process.

First, testers should analyze the transaction requirements and identify high-risk scenarios. Test cases should then be designed using Equivalence Partitioning, Boundary Value Analysis, Decision Tables, and State Transition Testing.

Important scenarios include:

- Successful transaction
- Insufficient balance
- Invalid account number
- Invalid PIN/authentication
- Maximum transaction limit
- Duplicate transaction
- Network failure
- Server failure
- Session timeout
- Transaction rollback

Integration testing should verify communication between the user interface, application server, APIs, and



database.

After a defect is fixed, testers should perform retesting and regression testing. Critical and repetitive tests can be automated.

Defects should be properly documented and historical defect data should be analyzed to identify recurring problems.

Thus, structured testing improves test coverage, consistency, defect detection, regression effectiveness, transaction accuracy, and reliability, reducing repeated failures in the banking system.